

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Шаблоны проектирования

по курсу: Технология программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2025

1 Вариант задания – предметная область и шаблон

Вариант 14

Предметная область: облачный сервис для хранения файлов (папки с файлами).

Шаблон проектирования: Итератор (Iterator).

2 Словесная постановка задачи

Создать простую модель облачного хранилища файлов. Есть коллекция файлов (папка в облаке). Для перебора файлов используется шаблон проектирования Iterator. Итератор позволяет последовательно получать файлы, не раскрывая внутреннее устройство коллекции.

Все методы выводят сообщения на экран для демонстрации работы

3 Диаграмма классов для шаблона проектирования

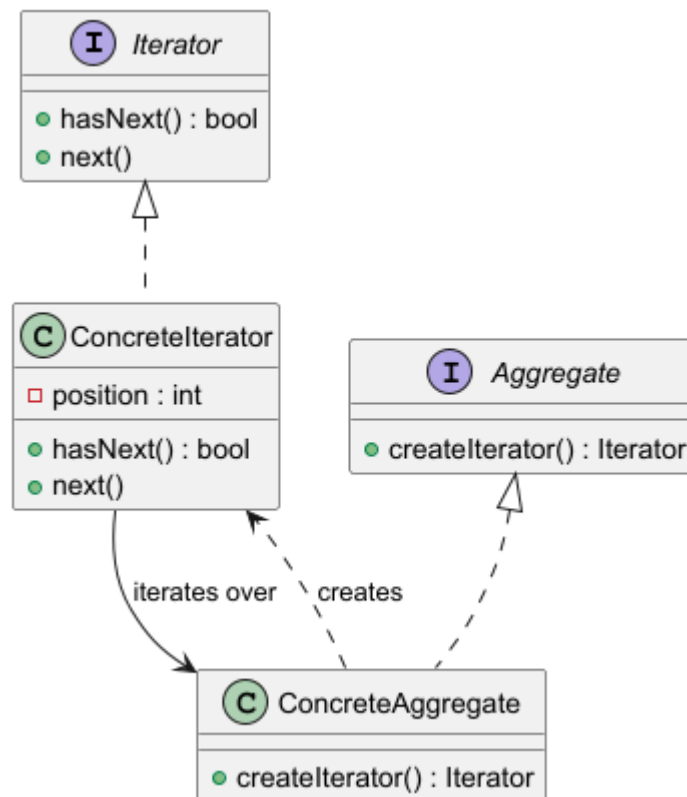


Рисунок 1 – Диаграмма классов шаблона

4 Диаграмма классов для подготовленной программы

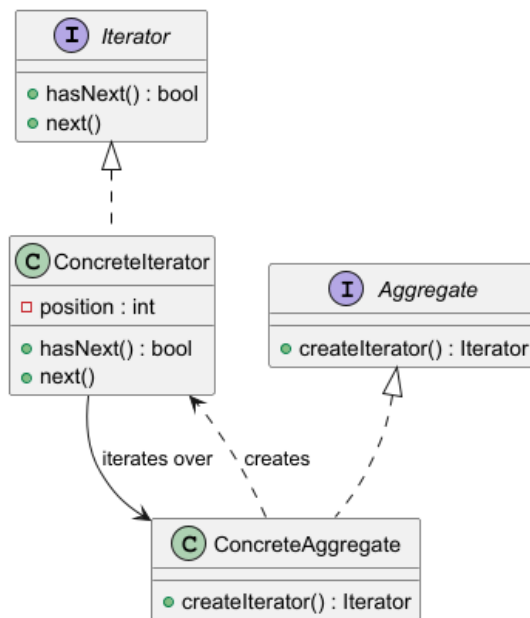


Рисунок 2 - Диаграмма классов программы

5 Диаграмма последовательности (взаимодействия) по алгоритмам программы

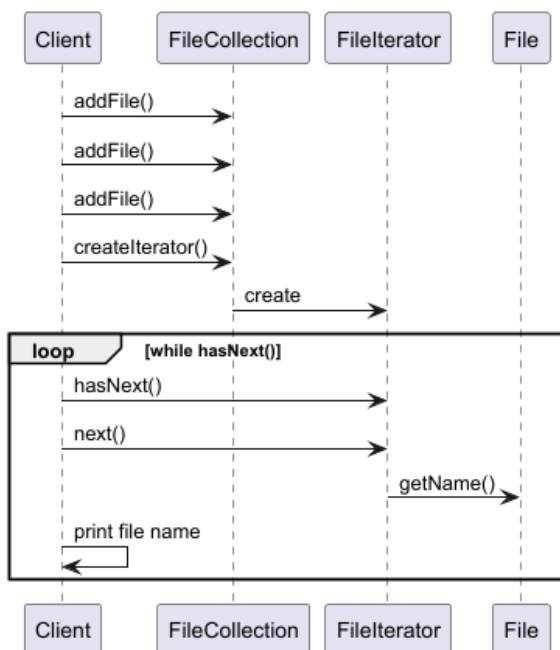
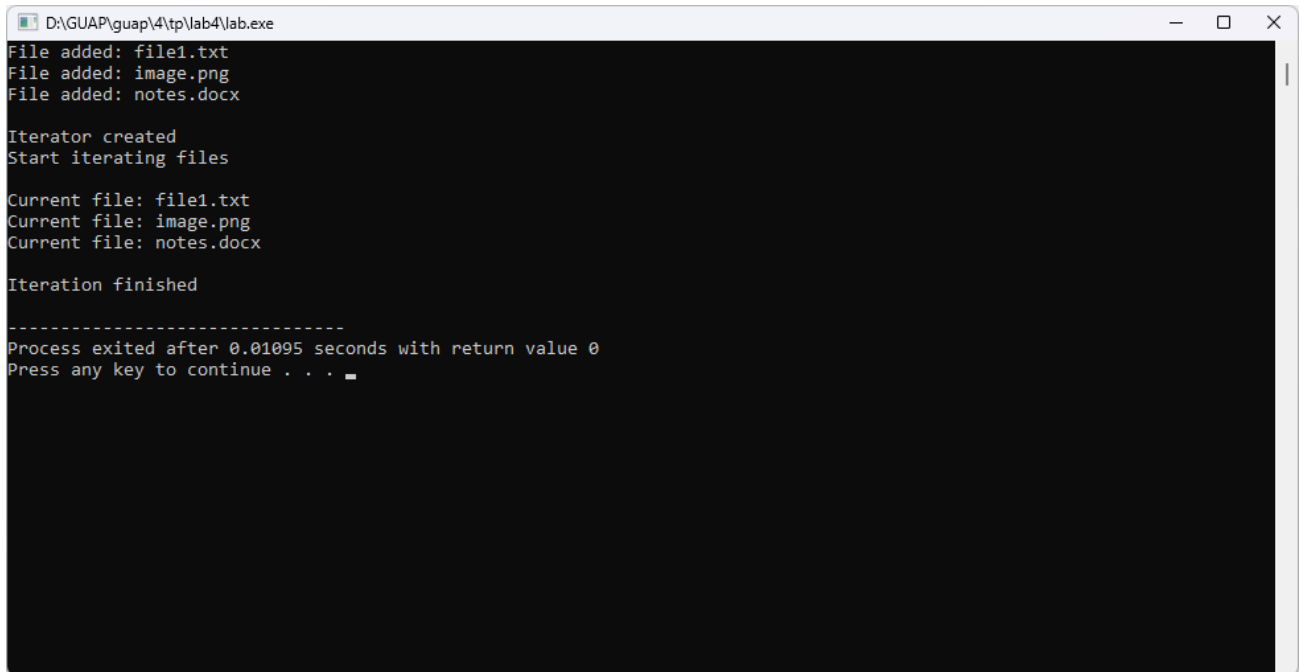


Рисунок 3 - Диаграмма последовательности программы

6 Примеры работы программы



```
D:\GUAP\guap\4\tp\lab4\lab.exe
File added: file1.txt
File added: image.png
File added: notes.docx

Iterator created
Start iterating files

Current file: file1.txt
Current file: image.png
Current file: notes.docx

Iteration finished

-----
Process exited after 0.01095 seconds with return value 0
Press any key to continue . . .
```

Рисунок 4 – Вывод в консоль

7 Листинг программы

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

/*
    Класс File — простой объект предметной области.
    Представляет файл, который хранится в облаке.
*/
class File {
private:
    string name;
public:
    File(string n) {
        name = n;
    }
}
```

```

/*
    Метод возвращает имя файла.
    Используется итератором и клиентским кодом.
*/
string getName() {
    return name;
}
};

/*
    Интерфейс Iterator.
    Описывает стандартные операции итератора:
    - проверка, есть ли следующий элемент
    - получение следующего элемента
*/
class Iterator {
public:
    /*
        Проверяет, остались ли элементы в коллекции.
        Возвращает true, если следующий элемент существует.
    */
    virtual bool hasNext() = 0;

    /*
        Возвращает следующий элемент коллекции.
        В нашем случае — указатель на объект File.
    */
    virtual File* next() = 0;
};

/*
    Интерфейс Aggregate (Коллекция).
    Любая коллекция должна уметь создавать итератор.
*/
class Aggregate {
public:
    /*
        Создаёт итератор для обхода коллекции.
    */
    virtual Iterator* createIterator() = 0;
};

```

```

/*
    Класс FileCollection — конкретная коллекция файлов.
    Представляет папку в облачном хранилище.
*/
class FileCollection : public Aggregate {
private:
    vector<File> files; // Список файлов (внутреннее хранилище)
public:
    /*
        Добавляет файл в коллекцию.
        Для наглядности выводит сообщение в консоль.
    */
    void addFile(File f) {
        files.push_back(f);
        cout << "File added: " << f.getName() << endl;
    }

    /*
        Возвращает количество файлов в коллекции.
        Используется итератором.
    */
    int getSize() {
        return files.size();
    }

    /*
        Возвращает файл по индексу.
        Итератор вызывает этот метод, чтобы получить текущий файл.
    */
    File* getFile(int index) {
        return &files[index];
    }

    /*
        Реализация метода интерфейса Aggregate.
        Создаёт и возвращает итератор для данной коллекции.
    */
    Iterator* createIterator();
};

/*
    Класс FileIterator — конкретный итератор.

```

Отвечает за пошаговый обход файлов в FileCollection.

```
*/  
class FileIterator : public Iterator {  
private:  
    FileCollection* collection; // Указатель на коллекцию  
    int index;                // Текущая позиция итератора  
public:  
    /*  
        Конструктор.  
        Получает коллекцию, по которой будет выполняться обход.  
        Начальная позиция — 0.  
    */  
    FileIterator(FileCollection* c) {  
        collection = c;  
        index = 0;  
    }  
  
    /*  
        Проверяет, есть ли ещё файлы для обхода.  
        Если индекс меньше размера коллекции — значит элементы есть.  
    */  
    bool hasNext() {  
        return index < collection->getSize();  
    }  
  
    /*  
        Возвращает текущий файл и сдвигает итератор вперёд.  
    */  
    File* next() {  
        File* f = collection->getFile(index);  
        index++;  
        return f;  
    }  
};  
  
/*  
    Реализация метода createIterator().  
    Создаёт объект FileIterator и возвращает его клиенту.  
*/  
Iterator* FileCollection::createIterator() {  
    cout << "\nIterator created" << endl;  
    return new FileIterator(this);  
}
```

```

}

/*
    Главная функция программы.
    Демонстрирует использование шаблона Iterator.
*/
int main() {
    // Создаём коллекцию файлов (облачную папку)
    FileCollection cloudFolder;

    // Добавляем файлы в коллекцию
    cloudFolder.addFile(File("file1.txt"));
    cloudFolder.addFile(File("image.png"));
    cloudFolder.addFile(File("notes.docx"));

    // Создаём итератор для обхода файлов
    Iterator* it = cloudFolder.createIterator();

    cout << "Start iterating files\n" << endl;

    // Обход коллекции с помощью итератора
    while (it->hasNext()) {
        File* f = it->next();
        cout << "Current file: " << f->getName() << endl;
    }

    cout << "\nIteration finished" << endl;

    // Освобождаем память, выделенную под итератор
    delete it;

    return 0;
}

```